

Automated Student Attendance System

AI-Suggested, Teacher-Verified Attendance from Classroom Photos
Human-in-the-Loop AI - Complete Project Guide & Viva Reference

Prepared September 2026 | PBL Project | All figures in this document are measured, reproducible results

Project at a glance

Live deployment	https://autoattendance-xjeg.onrender.com (Render, Docker, free tier)
Source code	github.com/AA7304-MEH/automated-attendance-system--pbl-
Core idea	One classroom photo -> every face detected & matched -> AI pre-fills attendance -> teacher confirms / corrects / assigns -> auditable register
Stack one-liner	Python + Flask + SQLAlchemy (SQLite/Postgres) + dlib face_recognition + OpenCV + Pillow + pandas + Gunicorn + Docker
Quality gates	61 automated checks passing (11 engine + 50 web) + 20/20 live deployment checks
Demo logins	Admin: teacher@college.edu / teacher123 Teacher: arjun@college.edu / teacher123 Student portal: roll no + PIN 1234

How to read this document

Sections 1-4 give the big picture (summary, problem, architecture, tech stack). Section 5 explains the AI engine in depth - the technical heart you must be able to defend. Section 6 walks through every single feature module. Sections 7-13 cover the database, routes, security, deployment and measured test results. Section 14 is a viva Q&A drill. Read 5, 6 and 14 twice before your presentation.

1. Executive Summary

Manual roll-call wastes five to ten minutes of every lecture, is prone to proxy (buddy) attendance, and produces registers that cannot be audited. This project replaces that ritual with a single photograph. A teacher uploads one picture of the classroom; a face-recognition engine detects every face, converts each into a 128-dimension mathematical signature, and compares the signatures against enrolled students. The result is not blind automation: every face gets a verdict in one of three confidence bands, and the teacher always makes the final decision on a visual verification screen.

The defining design decision is the Human-in-the-Loop (HITL) policy. High-confidence matches (distance 0.45 or lower) arrive pre-approved; medium-confidence matches (0.45-0.60) are shown side-by-side with the registered portrait for the teacher to confirm or correct; unmatched faces are marked Unknown and can only be assigned by a human. Every stored mark records HOW it was made (auto / review / manual / system-absent), the AI confidence percentage, and which teacher verified it - a fully auditable register.

Around that core, the system ships complete: role-based access (admin + teachers), live analytics, an at-risk alert dashboard with 7-day trends, CSV exports, scheduled email digests, a PIN-protected student self-service portal, hardened security (CSRF, rate limiting, hashed credentials, security headers), Docker deployment on Render, and 61 automated tests plus a 20-point live deployment verification - all passing.

2. Problem Statement & Objectives

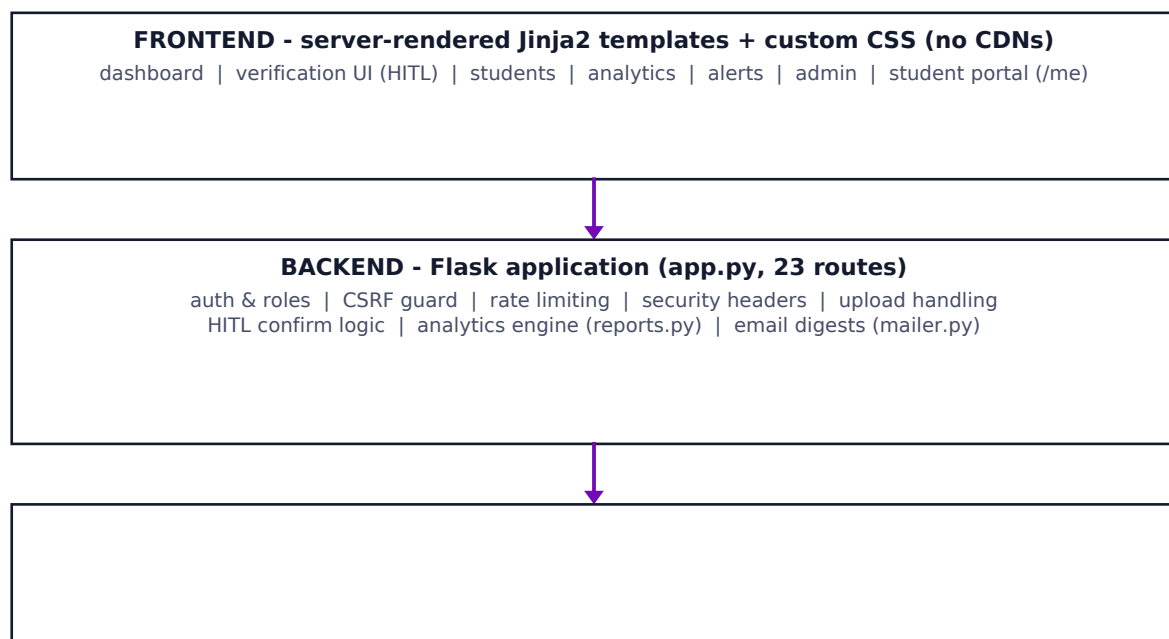
Problems addressed

- Roll-call consumes 5-10 minutes of every lecture (about 30+ hours per subject per semester).
- Proxy attendance: one student answering for an absent friend is undetectable on paper.
- Paper registers are error-prone, easy to lose, and impossible to audit after the fact.
- Students below the 75% attendance requirement are discovered too late - at exam form filling.

Objectives

- Mark attendance for an entire class from one photo, in seconds, from any phone browser.
- Keep the teacher as the final authority: the AI suggests, the human decides (HITL).
- Record provenance for every mark: method, AI confidence %, and verifying teacher.
- Surface at-risk students early via thresholds, 7-day trends, alerts and email digests.
- Protect people and data: hashed credentials, CSRF, rate limiting, no real biometrics in git.
- Deploy on commodity infrastructure (Docker, free tier) with zero paid dependencies.

3. System Architecture



AI CORE - face_engine/

detector.py (HOG detection + cropping) encoder.py (enrollment, 128-d encodings)

recognizer.py (matching + 3-band policy + annotated image output)

DATA LAYER

SQLite (Postgres-ready via DATABASE_URL): teachers, students, subjects,

attendance_sessions, attendance | face_encodings.pkl (biometric templates)

The browser talks only to Flask. Flask is the only writer of the database. The face engine is a pure Python library - the web layer never touches dlib directly, which means the engine can be swapped (e.g. InsightFace/YOLOv8) without changing a single route.

4. Complete Technology Stack

Layer	Technology	Why it is used / where
Language	Python 3.13 (3.11 in Docker)	AI/ML ecosystem, readability, team familiarity
Web framework	Flask 3.x + Jinja2	Lightweight routing + server-rendered templates; ideal for an AI-backed MVP
ORM / DB	Flask-SQLAlchemy 3.1 + SQLite (Postgres-ready)	Type-safe models, unique constraints, zero-config dev DB; DATABASE_URL swaps in hosted Postgres for scale
Authentication	Flask-Login + Werkzeug hashing	Session-based teacher auth; PBKDF2/scrypt password hashing (never plain text)
Face detection	dlib HOG detector (via face_recognition)	Fast CPU detection of frontal faces; runs on free-tier hardware
Face identity	dlib ResNet - 128-d embeddings	Deep metric embeddings: same person maps to nearby points in 128-d space
Matching	Euclidean distance (NumPy)	One vector subtraction per enrolled student - microseconds, no model call
Image processing	OpenCV 4.11 (headless in Docker)	Crop faces, draw annotated verdicts, encode JPEG outputs
Photo normalization	Pillow (EXIF + LANCZOS resize)	Fixes rotated phone photos (EXIF) and tames 12MP images for 512MB RAM
Exports / math	pandas	One-line CSV export of attendance and student summaries
Email	smtplib (starttls) + template	Console backend for demos, real SMTP via env vars; cron CLI included
Production server	Gunicorn (WSGI)	2-process-grade serving; worker count tuned via WEB_CONCURRENCY
Packaging	Docker + docker-compose	Same image everywhere; data/ volume persists DB + portraits
Hosting	Render.com (free tier)	Auto-builds the Dockerfile, public HTTPS URL, zero-cost demo
Frontend styling	Hand-written CSS3	Glassmorphism nav, gradient system, hover states - zero CDN dependency, works offline
Version control	Git + GitHub	Single source of truth; 8+ feature commits, verified remote sync

Why face_recognition (dlib) and not a face API? It runs 100% offline and free, is well-documented, and its distance metric gives us interpretable confidence bands - which is exactly what the Human-in-the-Loop policy needs.

5. The Face-Recognition Engine (Technical Heart)

5.1 The recognition pipeline

- STEP 1 - Detection (HOG): the image is scanned for face-like gradient patterns. Detected boxes are returned as (top, right, bottom, left). Fast on CPU; 'cnn' model available for GPU accuracy.
- STEP 2 - Embedding: each detected face is aligned and passed through a ResNet that outputs 128 numbers (the 'encoding'). Think of it as a fingerprint: photos of the same person land close together in 128-dimensional space regardless of lighting, angle or camera.
- STEP 3 - Matching: the encoding of each classroom face is compared with every enrolled encoding using Euclidean distance (plain vector distance). The smallest distance wins. This is microseconds of math.
- STEP 4 - Policy: the winning distance is mapped to one of three verdicts (below).
- STEP 5 - Output: the web layer receives structured results (id, distance, similarity %, verdict) plus a color-annotated copy of the photo (green / amber / red boxes) and face crops for the comparison UI.

5.2 The three-band Human-in-the-Loop policy

Distance d	Verdict	What the teacher sees	Stored method
$d \leq 0.45$	AUTO-APPROVED (green)	Pre-checked 'Present' card with similarity bar; teacher can still uncheck	auto
$0.45 < d \leq 0.60$	NEEDS REVIEW (amber)	Side-by-side: classroom face VS registered portrait; confirm or correct via dropdown	review (confirmed) / manual (corrected)
nobody ≤ 0.60	UNKNOWN (red)	Face crop with '?'; teacher assigns a roll number - or leaves them absent	manual / stays absent

Everyone the teacher does not mark is written as ABSENT (method='system'). One photo therefore produces a complete register - and a missed detection can never inflate attendance.

5.3 Measured engine results (all reproducible via scripts)

Probe	Result
Group photo: 6 students, 2 loose rows	6/6 faces detected and enrolled
Same photo augmented (mirrored, dimmed -18%, 0.85x scale, JPEG-70)	6/6 re-identified; distances 0.188-0.262 -> all auto-approved
Leave-one-out honesty control (store WITHOUT one student)	That student rejected at distance 0.726 (>> 0.60) - no false match
Stranger portrait (never enrolled)	distance 0.802 -> Unknown; never auto-approved
Stress test (0.55x size + blur + underexposure + JPEG-45)	2 detectable faces at 0.470 / 0.583 -> correctly NEEDS REVIEW; 0 false auto-approvals
Simulated phone photo (pixels rotated 90 + EXIF flag)	Before fix: 0/6 detected. After EXIF normalization: 6/6 auto-approved

Blueprint correction we defend in the viva: the original plan auto-approved at 'confidence $\geq 85\%$ ' computed as $1 - \text{distance}$, i.e. distance ≤ 0.15 . Real same-person dlib distances land at 0.20-0.45, so that rule would send EVERY genuine student to manual review. Our bands (0.45 / 0.60) follow dlib conventions and are validated by the probes above.

5.4 Real-world photo hardening (EXIF + resolution)

Phone cameras store pixels sideways and add an EXIF flag telling viewers to rotate. OpenCV and dlib IGNORE that flag - so an unprocessed phone upload presents sideways faces and detection fails (proven: 0/6). Every upload is therefore normalized with Pillow: EXIF-transpose to true upright, convert to RGB, and downscale to 2000px on the long side (1600 for enrollment portraits) using Lanczos resampling. This also keeps 12MP photos fast and memory-safe on the 512MB free tier.

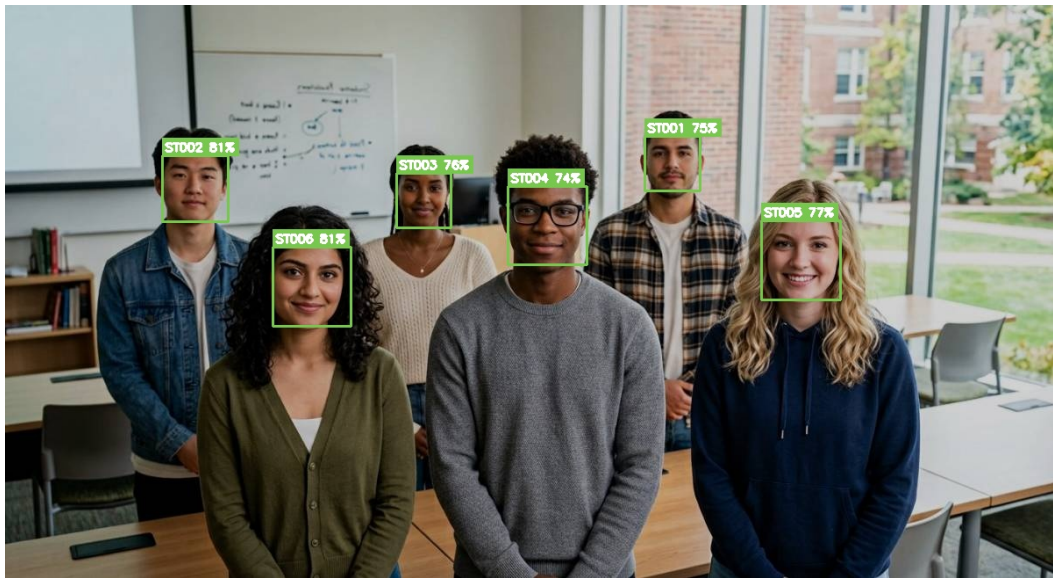


Figure 1 - Good capture: all six students detected, matched and auto-approved (green).

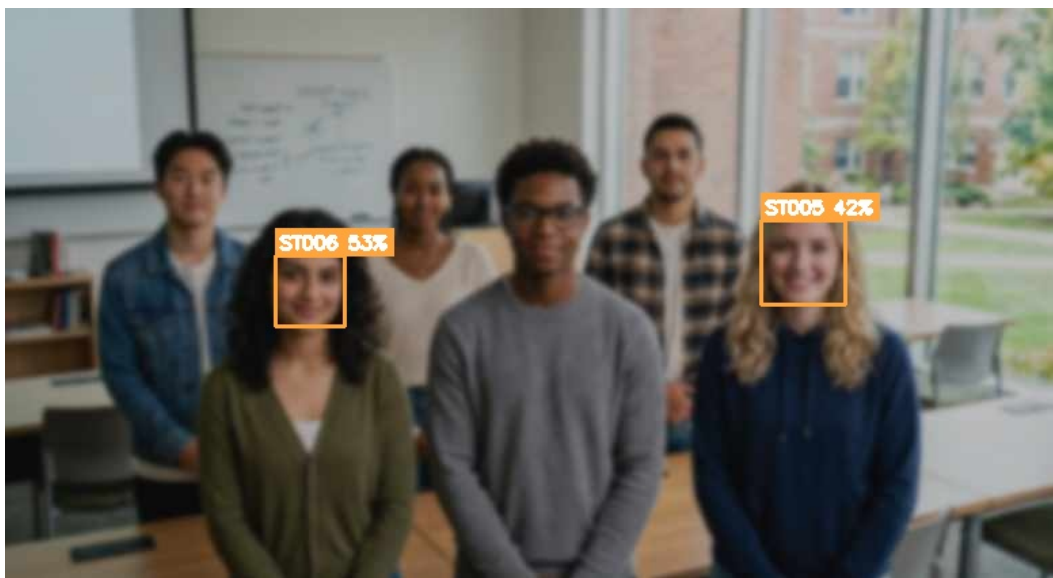


Figure 2 - Degraded capture: only 2 faces detectable, both correctly routed to NEEDS REVIEW (amber).

6. Complete Feature Walkthrough (Every Module)

6.1 Authentication & roles

Teachers log in with email + password (Werkzeug-hashed). Two roles: ADMIN (full visibility, manages accounts and subjects) and TEACHER (sees and marks only assigned subjects and own sessions). Failed logins are rate-limited per IP: 5 failures per 60 seconds. Where: app.py login/admin routes, database/models.py Teacher.role. Demo: log in as arjun@college.edu - Admin page returns 403.

6.2 Teacher dashboard

Landing page after login: today's present count across your subjects, subject list, the 75% at-risk line, and the 8 most recent sessions with verified/pending status. The photo upload form lives here - including the one-click 'Try the demo photo' button that runs the bundled classroom shot without needing a file.

6.3 Photo upload + AI pass (POST /take_attendance)

Validates extension (.jpg/.jpeg/.png) and 16MB cap, saves with a random UUID name, then normalizes (EXIF transpose, RGB, downscale). The engine detects and matches every face; results are persisted as a PENDING AttendanceSession with a JSON payload; face crops and a color-annotated photo are written for the verification screen. Duplicate detections of the same student are deduplicated (best kept). Zero faces -> friendly error, nothing saved.

6.4 Verification page - the HITL core (GET /session/<id>/verify)

- Summary chips: counts of auto-approved / needs-review / unknown faces.
- 'What the AI saw': the annotated classroom photo with color-coded boxes and confidence labels.
- GREEN cards: pre-checked Present, name, roll, similarity bar, percentage. Teacher may uncheck.
- AMBER cards: side-by-side VS comparison (classroom crop vs registered portrait), similarity bar, and an identity dropdown preselected to the AI's guess - changing it CORRECTS the AI and is stored as manual.
- RED cards: unknown face crop + dropdown of students not yet matched; assign or leave absent.
- 'Also present?' multiselect: roster students the photo missed (small/blurry faces) can be marked present.
- Sticky confirm bar warns: everyone unmarked will be recorded ABSENT for this subject.

6.5 Confirmation & the auditable register (POST /session/<id>/confirm)

The teacher's decisions are written transactionally: identities confirmed as the AI guessed store method 'auto' or 'review' with the confidence %; any changed identity stores 'manual'; the extra-present list stores 'manual'; the entire remaining roster stores 'Absent' with method 'system'. Every row records verified_by (which teacher) and links to the session. Upsert per (student, subject, date) means re-taking attendance the same day updates rather than duplicates.

6.6 Sessions & provenance

Each upload is a session: subject, teacher, timestamp, the original photo, the AI's JSON verdicts, status pending -> verified, and decision time. The session detail page shows the annotated photo plus present/absent breakdown with per-student method chips (auto 76% / review 53% / manual). This is what makes the register auditable months later.

6.7 Student enrollment & management (/students)

Enroll a student with roll number, name, department, semester, optional portal PIN, and a portrait. The portrait is normalized (EXIF + resize to 1600px) and its face encoding is computed immediately - enrollment fails with a clear error if no single face is found. The roster table shows each student's live attendance percentage with color-graded bars. Re-enrolling replaces the old encoding. CLI alternative: scripts/enroll_students.py --rebuild / --add photo.jpg ST042 / --list.

6.8 Analytics (/analytics)

Overall attendance donut, today's donut, at-risk counter, a daily trend line chart (pure SVG - renders anywhere, even offline) with a red dashed at-risk line at 75%, per-student bars, and per-subject bars. Color language everywhere: green $\geq 75\%$, amber 50-74%, red $< 50\%$. CSV export buttons at the top.

6.9 Alerts dashboard (/alerts)

The teacher's daily action list. AT-RISK table ($< 75\%$): student, overall %, last-7-days %, trend arrow (up = improving, down = declining, computed as recent minus overall), and each student's WORST subject computed over subjects with ≥ 3 classes. BORDERLINE table ($< 85\%$) for early intervention. Feed for the email digest and CSV summary - all three views share one implementation (reports.py).

6.10 CSV exports

Two downloads, both pandas-generated: attendance_raw.csv (every mark: date, time, student, subject, status, method, AI confidence, verified by) and students_summary.csv (per-student totals, percentage, last-7-day percentage, trend delta, worst subject, at-risk flag). Ready for the college office or Excel.

6.11 Email digest (/alerts/email + scripts/send_alerts.py)

An HTML email (branded header, at-risk table with trend arrows, borderline watch list, suggested action) sent to all teachers. Two backends: CONSOLE (default; renders the exact email into data/outputs/emails/ for demos - preview it at /alerts/email/preview) and SMTP (set EMAIL_BACKEND=smtp + SMTP_HOST/USER/PASSWORD env vars). Production cron: '0 8 * * * python scripts/send_alerts.py'.

6.12 Admin panel (/admin, admin-only)

Create teacher/admin accounts (name, email, password ≥ 6 chars, role), add subjects, and reassign subject ownership inline. Subject visibility and session ownership are enforced server-side - teachers get 403 on anything not theirs. Demo seed: admin teacher@college.edu owns CS201+CS302; Prof. Arjun Nair owns CS405.

6.13 Student self-service portal (/me, public)

No staff login: students enter roll number + PIN (stored hashed; demo PIN 1234, set at enrollment or backfilled). They see their attendance donut, subject-wise bars, recent history, and a prominent warning when below 75%. Errors are generic (no user enumeration) and attempts share the login rate limiter. Read-only by design: disputes go to the teacher, who owns the data.

6.14 Security model (cross-cutting)

- CSRF token required on every POST (before_request guard) - blocks cross-site form attacks.
- Security headers on every response: Content-Security-Policy (self only), X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Referrer-Policy: same-origin.
- Login + portal rate limiting: 5 failures / 60s per IP (in-memory; Redis recommended at scale).
- Session cookies HttpOnly + SameSite=Lax; 16MB upload cap; file extension allow-list.
- Media routes are login-gated and regex-validated (path traversal like ../../etc/passwd is blocked).
- Passwords and PINs hashed (Werkzeug scrypt/PBKDF2). Biometric encodings live in ONE pickle owned by the engine - the relational DB stores no biometrics. Repo contains only AI-generated faces.

6.15 Deployment (Docker + Render)

Dockerfile: python3.11-slim + dlib-bin (prebuilt wheel - no compile) + explicit deps; face_recognition installed --no-deps on top (its transitive deps Pillow/Click/models listed manually); gunicorn started via WEB_CONCURRENCY (default 1 - safe for 512MB). docker-compose mounts ./data for persistence. Render.com builds the image automatically; first boot creates the schema, seeds demo data (race-safe via file lock + IntegrityError fallback after we fixed a real two-worker crash), and rebuilds the encoding

DB. Free tier: sleeps after ~15 min idle (~50s wake); redeploys reset data/ but auto-reseed.

7. Database Design (5 tables)

Table	Columns & purpose
teachers	id PK name email UNIQUE password_hash (scrypt) role ('admin' 'teacher') - account + role
students	id PK student_id UNIQUE (roll, e.g. ST001) first/last name email department semester photo_path (portrait filename) portal_pin (hashed) active - roster + portal login
subjects	id PK name code UNIQUE (CS201...) teacher_id FK - course catalogue + ownership
attendance_sessions	id PK subject_id FK teacher_id FK photo_path results_json (AI verdicts) status pending->verified created_at decided_at - one upload = one AI run + decision
attendance	id PK session_id FK student_id FK subject_id FK date time status Present/Absent confidence FLOAT (AI %) method auto/review/manual/system verified_by FK created_at. UNIQUE(student, subject, date) -> same-day re-remarks UPDATE, never duplicate

Design decision to defend: biometric encodings are NOT in the database. One writer owns them (face_engine's face_encodings.pkl), eliminating two-source-of-truth sync bugs. The DB stores profile data + photo references. Migrations for older DBs are handled in-place (ALTER TABLE + backfill).

8. Complete Route Map (23 endpoints)

Method	Route	Purpose	Access
GET	/	Redirect to dashboard or login	public
GET/POST	/login	Login form + authentication (rate-limited)	public
GET	/logout	Clear session	logged in
GET	/teacher/dashboard	Stats, subjects, upload form, recent sessions	teacher (filtered)
POST	/take_attendance	Upload/normalize photo -> AI pass -> pending session	owner teacher
GET	/session/<id>/verify	HITL verification page (3-band UI)	owner/admin
POST	/session/<id>/confirm	Write final register with provenance	owner/admin
GET	/session/<id>	Session detail + annotated photo	owner/admin
GET/POST	/students	Roster + enroll (encodes face live)	any staff
GET	/analytics	Donuts, SVG trend, per-student/subject bars	any staff
GET	/alerts	At-risk + borderline tables with trends	any staff
GET	/alerts/email/preview	Render the exact digest email	any staff
POST	/alerts/email/send	Send digest (console or SMTP)	any staff
GET	/export/attendance.csv	Raw mark log with provenance	any staff
GET	/export/students.csv	Per-student summary + risk flag	any staff
GET	/admin	Accounts + subjects management	ADMIN only (403 else)
POST	/admin/teachers/add	Create teacher/admin account	ADMIN
POST	/admin/subjects/add	Create subject	ADMIN
POST	/admin/subjects/<id>/assign	Reassign subject to teacher	ADMIN

GET/POST	/me	Student portal: roll + PIN -> public (PIN) personal attendance	
GET	/student?student_id=	Staff-side student lookup	any staff
GET	/media/<sid>/<name>	Session crops/annotated image	logged in, regex-guarded
GET	/photo/<name>	Enrollment portraits	logged in, regex-guarded

9. Project Structure (what every file does)

```

attendance-system/
├─ app.py           Flask app: 23 routes, HITL confirm logic, analytics, roles, portal, security
├─ config.py        Single source of truth: bands (0.45/0.60), 75% threshold, rate limits, paths, EMAIL/DATABASE
envs
├─ face_engine/
│  ├─ detector.py    HOG detection -> DetectedFace(location, encoding); crop_face with margin
│  ├─ encoder.py     FaceEncodingStore: enroll (dominant-face rule, replace-on-re-enroll), rebuild, pickle I/O
│  └─ recognizer.py  FaceRecognizer.recognize() -> 3-band verdicts; .annotate() -> color-coded JPEG
├─ database/
│  ├─ models.py      5 SQLAlchemy models (teachers/students/subjects/sessions/attendance)
│  └─ seed.py        Idempotent demo seed + in-place migrations + race-safe file lock
├─ reports.py       ONE implementation of per-student stats (overall, 7-day trend, worst subject) shared by
UI/CSV/email
├─ mailer.py        Digest builder + console/SMTP backends
├─ templates/ (10 pages) base(glass nav) · login · dashboard · verify(HITL) · session · students · analytics · alerts ·
admin · portal · email
├─ static/style.css  Full design system: gradients, glass, cards, chips, bars, responsive hamburger
├─ scripts/
│  ├─ setup_env.sh   One-command env restore (prebuilt dlib strategy, ~30s)
│  ├─ enroll_students.py Enrollment CLI (--rebuild / --add / --list)
│  ├─ run_phase2_demo.py End-to-end engine demo + honesty probes
│  ├─ stress_test.py  Degraded-photo probe (proves the review band)
│  ├─ send_alerts.py  Cron email sender (--preview)
│  ├─ demo_photo.py   Recognize any photo from the CLI (table + annotation)
│  └─ make_project_pdf.py Regenerates this very document
├─ tests/
│  ├─ test_face_engine.py 11 engine checks (incl. stranger rejection, augmentation)
│  └─ test_web_flow.py    50 web checks (CSRF, all 3 HITL branches, roles, 403s, email, rate limit, traversal)
├─ docs/             PROJECT_REPORT.md (write-up) · DEMO.md (10-min viva script) · DEPLOY.md (Render guide)
├─ Dockerfile         python3.11-slim + dlib-bin + gunicorn (WEB_CONCURRENCY aware)
├─ docker-compose.yml Port 8000 + ./data volume
└─ data/              student_faces/ (portraits) · uploads/ · outputs/ (annotated) · face_encodings.pkl · attendance.db
  
```

10. Testing & Quality Evidence

Suite / probe	Count	What it proves
tests/test_face_engine.py	11 PASS	Enrollment integrity (128-d), re-identification, augmentation robustness, stranger rejection
tests/test_web_flow.py	50 PASS	CSRF, auth, all 3 HITL branches, absent-by-default, correction->manual, enrollment, analytics, CSVs, alerts, email, roles+403s, portal, media ACL, traversal, headers, rate limit
Live deployment smoke (2026-09-21)	20/20 PASS	Every page, the full AI flow, exports, roles and portal on the production Render URL
Accuracy probes (leave-one-out, stranger, stress)	5 probes	0.188-0.262 same-person 0.726 rejection 0.802 stranger 0 false auto-approvals
EXIF phone-photo probe	0/6 -> 6/6	Real-world rotation handling works

Everything is reproducible: `python tests/test_web_flow.py` | `pytest tests/ -v` | `python scripts/run_phase2_demo.py` | `python scripts/stress_test.py` | `python scripts/demo_photo.py`
`data/uploads/classroom_demo_hard.jpg`

11. How to Run It

Local: `bash scripts/setup_env.sh -> python app.py -> http://localhost:8000`

Docker: `docker compose up --build`

Cloud: already live at <https://autoattendance-xjeg.onrender.com> (Render auto-builds from GitHub)

Enroll real people: Students -> Enroll (portrait: one person, face ~1/3 of frame, good light) -> take a group photo -> Dashboard -> upload -> verify -> confirm.

12. Viva Q&A Drill (likely questions, ready answers)

Question	Your answer
How does the AI recognize a face?	HOG finds the face; a ResNet converts it to 128 numbers; we compare those numbers to every enrolled student with Euclidean distance and take the closest.
What is a 128-d encoding?	A deep-metric embedding: a point in 128-dimensional space where the same person's photos cluster tightly regardless of lighting/angle - a mathematical fingerprint.
Why distance and not a percentage from a classifier?	Distance is interpretable and thresholdable: we can define honest bands (0.45 auto / 0.60 review) and route uncertainty to a human instead of guessing.
Why did you change the blueprint's 85% threshold?	85% as 1-distance means distance 0.15 - genuine matches never get that close (ours: 0.19-0.26), so every student would need manual review. We validated bands empirically instead.
What stops the AI marking a random person present?	Three layers: the 0.60 tolerance (strangers measured at 0.80), the red Unknown flow requiring a human to assign, and absent-by-default for undetected students. Leave-one-out test proves rejection at 0.726.
Two students look alike - what happens?	Their distance lands in the amber band -> teacher reviews side-by-side and corrects the dropdown; the correction is stored as method=manual, so we even know when the AI erred.
Can someone cheat with a printed photo?	A 2-D photo can fool pure matching - honest limitation. Production adds liveness detection (blink/depth). Documented in the report's future work.
What if the classroom photo is blurry/far?	Some faces won't be detected -> they default to Absent (never wrongly present) and the teacher uses the 'Also present?' list. Degraded probes never auto-approved anything.
Why SQLite? Why not MySQL?	Zero-config for the demo and perfectly fast at college scale; the SQLAlchemy layer + DATABASE_URL mean hosted Postgres is a config change, not a rewrite.
What is CSRF and how did you stop it?	Cross-Site Request Forgery - attacker form posts as a logged-in user. Every POST must carry a session token our templates embed; requests without it get 400.
How do you protect passwords?	Werkzeug scrypt/PBKDF2 hashing with salts - never reversible, never logged. Portal PINs hashed the same way.
Why is attendance data trustworthy?	Every mark stores method (auto/review/manual/system), AI confidence %, verifying teacher, and links to the session with the annotated photo - a full audit trail.
How does it handle real phone photos?	EXIF orientation auto-rotation + downscaling on upload; proven by simulation: 0/6 detection before the fix, 6/6 after.
How would you scale to 500 students?	Vectorized distance already handles thousands of encodings in microseconds; move DB to Postgres (config-only), swap HOG for a GPU/CNN detector or InsightFace, cache the encoding matrix, add background workers for email.
Is face data safe here?	Repo contains only AI-generated faces; real biometrics stay in a git-ignored volume. Production use requires consent + retention policy (DPDP Act compliance) - stated in the report.
What was the hardest bug?	Two unicorn workers seeding the empty DB simultaneously on first deploy - UNIQUE constraint crash. Fixed with an exclusive file lock + fallback; then verified with a fresh-DB deploy.

13. Limitations & Future Work

- Detection ceiling: extreme distance/blur drops faces (absent-by-default + manual list mitigates).
- No liveness/anti-spoofing yet (printed-photo attack) - blink or depth-based liveness is future work.
- Free tier: sleeps after idle, ephemeral disk on redeploys (auto-reseed keeps demos working).
- Accuracy on larger, real-world cohorts should be re-validated (bias/lighting audits) before production.
- Roadmap: InsightFace/YOLOv8 engine behind the same interface, OTP-based student identity, parent email alerts, admin PIN-reset UI, timetable-aware absent messaging.

One-line summary for your viva: 'The AI does in three seconds what roll-call does in five minutes - and a teacher, not an algorithm, signs every single mark.'